

Numerical solution of the two-dimensional heat equation

David v. Rudisill

Project advisor: Dr. Patrick Shields

Dr. David Reed, Math/CS 481

Final Draft

April 19, 2012

Introduction and Background

Partial differential equations arise in a large number of fields in science and engineering, ranging from weather modelling to cosmology, and generally such equations cannot be solved analytically or by qualitative methods. Thus the solutions to such equations must be obtained numerically. With the advent of high-speed digital computers, numerical methods have offered insights into equations that have otherwise proven to be intractably difficult to solve, and hence our understanding of physical problems associated with such equations has expanded greatly.

Numerical methods for solving differential equations of any type, whether ordinary or partial, essentially replace continuous derivatives with finite differences in which an independent variable is advanced a step-size h , e.g.,

$$\frac{\partial u}{\partial x} = \frac{1}{h}[u(x+h, t) - u(x, t)] + E, \quad (1)$$

where the error term E can be calculated using Taylor series expansions. Replacing derivatives with finite differences almost always produces a non-zero error term, and in the case of equation (1) the error is $O(h)$. For any given derivative, there exists many finite-difference formulas that approximate the value of the derivative, and which difference formula is used depends on the magnitude of the error deemed acceptable. For example, a finite-difference formula more accurate than equation (1) is the central difference formula,

$$\frac{\partial u}{\partial x} = \frac{1}{2h}[u(x+h, t) - u(x-h, t)] + E, \quad (2)$$

where $E = O(h^2)$, a substantial improvement over the error of equation (1).

Consider the following partial differential equation,

$$\begin{aligned} \frac{\partial u}{\partial x} &= -\frac{\partial u}{\partial t}, \\ u(0, 0) &= u(a, b) = 0. \end{aligned} \quad (3)$$

A simple routine to solve this equation on a computer is to replace the derivatives with the finite-difference formula in equation (2),

$$\frac{1}{2h}[u(x+h, t) - u(x-h, t)] = -\frac{1}{2h}[u(x, t+k) - u(x, t-k)], \quad (4)$$

where the (possibly different) step sizes h and k can be incremented. With a relation such as equation (4), the values of $u(x, t)$ can be computed on the grid (ih, jk) , where $a = nh$ and $b = mk$ for some integers n and m . If the errors are carefully controlled, we can expect that the computed values of $u(x, y)$ correspond accurately to the values of the analytic solution. Otherwise, if the error grows beyond acceptable bounds, the values of the numerical solution may be inaccurate and correspond poorly to the values of the analytic solution.

The physical process we wish to study in this paper is heat conduction in a three-dimensional slab, and as a simplifying assumption we assume that no heat transfer occurs perpendicular to the xy -plane. Such a process is governed by the two-dimensional heat equation,

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \alpha \frac{\partial u}{\partial t}, \quad (5)$$

where $\alpha > 0$. The numerical method that we'll use for solving this equation is a two-dimensional version of the Crank-Nicholson method, which for the one-dimensional heat equation reads (Cheney, Kincaid, 588),

$$\frac{1}{h^2}[u(x+h, t) - 2u(x, t) + u(x-h, t)] = \frac{1}{k}[u(x, t) - u(x, t-k)], \quad (6)$$

with step sizes h and k . The left side of equation (6) expresses a second-order partial derivative in terms of a finite-difference formula; our first task will be to similarly express the Laplacian operator in equation (5) with a finite-difference formula. Once this is done, we arrive at a tridiagonal system associated with the two-dimensional heat equation. Using a computer routine, we then solve this system to obtain the values of $u(x, y, t)$ at grid points (ih, jk) . Finally, we conclude with an analysis of how accurate the numerical solution is.

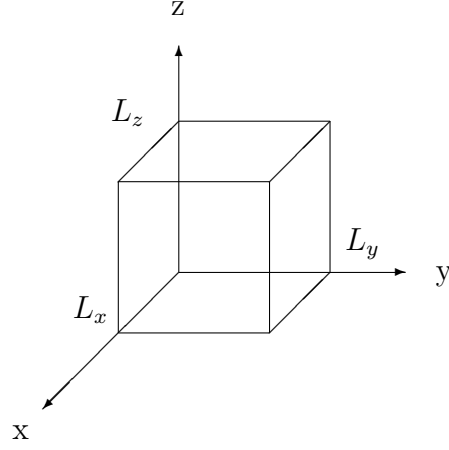
Determining the Associated Tridiagonal System

We consider a three-dimensional slab with sides having lengths L_x, L_y and L_z as in Figure 1. Our mathematical model is given by the heat equation in two-dimensions as well as the following boundary conditions,

$$\nabla^2 u = \alpha \frac{\partial u}{\partial t} \quad (7a)$$

$$u(0, y, t) = T_1, \quad u(L_x, y, t) = T_3, \quad u(x, y, 0) = T_2, \quad (7b)$$

Figure 1



where $T_1 > T_2 > T_3$. For simplicity we assume that no heat flow occurs on the the xy-planes at $z = 0$ and $z = L_z$, i.e.,

$$\frac{\partial u}{\partial z} = 0. \quad (8)$$

To implement the Crank-Nicholson method in two dimensions, it will be helpful to first calculate the Taylor series expansions for $f(x + h, y + h)$ and $f(x - h, y - h)$. We have,

$$\begin{aligned} f(x + h, y + h) = & f(x, y) + \frac{\partial f}{\partial x}h + \frac{\partial f}{\partial y}h + \frac{\partial^2 f}{\partial x \partial y}h^2 \\ & + \frac{1}{2} \left[\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \right] h^2 + \frac{1}{2} \left[\frac{\partial^3 f}{\partial x^2 \partial y} + \frac{\partial^3 f}{\partial x \partial y^2} \right] h^3 + \frac{1}{4} \frac{\partial^4 f}{\partial^2 x \partial^2 y} h^4 + \dots \end{aligned} \quad (9)$$

,
and,

$$\begin{aligned} f(x - h, y - h) = & f(x, y) - \frac{\partial f}{\partial x}h - \frac{\partial f}{\partial y}h + \frac{\partial^2 f}{\partial x \partial y}h^2 \\ & + \frac{1}{2} \left[\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \right] h^2 - \frac{1}{2} \left[\frac{\partial^3 f}{\partial x^2 \partial y} + \frac{\partial^3 f}{\partial x \partial y^2} \right] h^3 + \frac{1}{4} \frac{\partial^4 f}{\partial^2 x \partial^2 y} h^4 + \dots \end{aligned} \quad (10)$$

,
If we add equations (9) and (10) , we obtain,

$$\begin{aligned}
f(x+h, y+h) + f(x-h, y-h) &= 2f(x, y) + 2\frac{\partial^2 f}{\partial x \partial y} h^2 \\
&+ \left[\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \right] h^2 + \frac{1}{2} \frac{\partial^4 f}{\partial^2 x \partial^2 y} h^4 + \dots
\end{aligned} \tag{11}$$

Rearranging, we have,

$$\begin{aligned}
\nabla^2 &= \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \\
&\approx \frac{1}{h^2} \left[f(x+h, y+h) + f(x-h, y-h) - 2f(x, y) - 2h^2 \frac{\partial^2 f}{\partial x \partial y} \right],
\end{aligned} \tag{12}$$

where the error is $O(h^4)$.

We can approximate the mixed partial derviative in equation (12) as follows,

$$\begin{aligned}
\frac{\partial f}{\partial x} &\approx \frac{1}{h} [f(x+h, y) - f(x, y)], \\
\frac{\partial f}{\partial y} &\approx \frac{1}{h} [f(x, y+h) - f(x, y)] = g(x, y), \\
\frac{\partial^2 f}{\partial x \partial y} &\approx \frac{1}{h} [g(x+h, y) - g(x, y)] \\
&= \frac{1}{h^2} [f(x+h, y+h) - f(x+h, y) - f(x, y+h) + f(x, y)].
\end{aligned} \tag{13}$$

To find the error in equation (13), we first compute the Taylor series for $f(x, y+h)$ and $f(x+h, y)$,

$$f(x+h, y) = f(x, y) + \frac{\partial f}{\partial x} h + \frac{1}{2} \frac{\partial^2 f}{\partial x^2} h^2 + \frac{1}{6} \frac{\partial^3 f}{\partial x^3} h^3 + \dots \tag{14a}$$

$$f(x, y+h) = f(x, y) + \frac{\partial f}{\partial y} h + \frac{1}{2} \frac{\partial^2 f}{\partial y^2} h^2 + \frac{1}{6} \frac{\partial^3 f}{\partial y^3} h^3 + \dots \tag{14b}$$

If we substitute equations (14a), (14b) and (9) into equation (13), we find that

$$E = -\frac{1}{2} \left[\frac{\partial^3 f}{\partial x \partial y^2} + \frac{\partial^3 f}{\partial x^2 \partial y} \right] h - \frac{1}{4} \frac{\partial^4 f}{\partial x^2 \partial y^2} h^2 - \dots \tag{15}$$

Thus the error in equation (13) is $O(h)$. Likewise, the error in the expression,

$$2h^2 \frac{\partial^2 f}{\partial x \partial y} \approx 2[f(x+h, y+h) - f(x+h, y) - f(x, y+h) + f(x, y)], \quad (16)$$

is also $O(h)$. Substituting equation (16) into equation (12), we have,

$$\begin{aligned} \nabla^2 &= \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \\ &\approx \frac{1}{h^2} [-f(x+h, y+h) + 2[f(x+h, y) + f(x, y+h)] \\ &\quad - 4f(x, y) + f(x-h, y-h)]. \end{aligned} \quad (17)$$

Since we've expressed the Laplacian in terms of a finite-difference in equation (17), we can now write equation (7a) in terms of finite-differences,

$$\begin{aligned} \frac{1}{h^2} [u(x-h, y-h, t) - 4u(x, y, t) + 2[u(x+h, y, t) + u(x, y+h, t)] - \\ u(x+h, y+h, t)] = \frac{\alpha}{k} [u(x, y, t) - u(x, y, t-k)], \end{aligned} \quad (18)$$

where h and k are possibly different step sizes. We can simplify equation (18) with the following notation,

$$\begin{aligned} \beta &= \frac{k}{\alpha h^2}, \\ \gamma &= 4 + \frac{1}{\beta}. \end{aligned}$$

If we rearrange equation (18) so that the terms involving t and $t-k$ are on different sides of the equation, we have,

$$\begin{aligned} \beta [-u(x-h, y-h, t) + \gamma u(x, y, t) + u(x+h, y+h, t)] \\ - 2\beta [u(x+h, y, t) + u(x, y+h, t)] = u(x, y, t-k). \end{aligned} \quad (19)$$

In order to place equation (19) into a form that can be expressed as a tridiagonal system, we need to express $u(x+h, y, t)$ and $u(x, y+h, t)$ in terms of $u(x, y, t)$. If we set,

$$u(x+h, y, t) + u(x, y+h, t) \approx 2u(x, y, t), \quad (20)$$

where $E = O(h)$, we find that,

$$\begin{aligned} \beta[-u(x-h, y-h, t) + (\gamma-4)u(x, y, t) + u(x+h, y+h, t)] \\ = u(x, y, t-k). \end{aligned} \quad (21)$$

We can introduce column vectors **a** and **b** with components,

$$\begin{aligned} a_i &= u(ih, ih, t), \quad nh = L_x = L_y = L_z, \\ b_i &= u(ih, ih, t-k). \end{aligned}$$

We assume that the quantities b_i are known and that these values are used to calculate the values of a_i . With this notation, equation (21) can be written in the following matrix form,

$$\begin{pmatrix} 1 & \beta & 0 & \cdots & 0 & 0 & 0 \\ -\beta & 1 & \beta & \cdots & 0 & 0 & 0 \\ 0 & -\beta & 1 & \beta & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & -\beta & 1 & \beta \\ 0 & 0 & 0 & \cdots & 0 & -\beta & 1 \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ \vdots \\ b_{n-1} \end{pmatrix}.$$

Or, in block notation,

$$\mathbf{A}\mathbf{a} = \mathbf{b}. \quad (22)$$

This is the tridiagonal system associated with equation (7a). Using a computer routine, this system of equations can be solved to obtain the values of $u(x, y, t)$ at the grid points (ih, ih, jk) .

Solution of the Associated Tridiagonal System

The computer code used for solving the tridiagonal system (22) is shown below (Cheney, Kincaid, 589). The coefficient matrix in equation (22) contains three non-zero bands along the diagonal; the **main** function treats each band as a column vector. In the code below, all entries of column vector **C** are assigned the value 1 and the entries of column vector **D** are assigned the values β . The **main** function also initializes the column vector **U** by using the initial condition $u(x, y, 0) = 0$.

Once this is done, **main** calls the function **Tri** to solve the tridiagonal system for $t = 0$. The outer **j**-loop advances the time step with step-size k , and each time through the loop the column vector **U** with entries $u(ih, ih, jk)$

is computed by `main` calling the `Tri` function. Thus to compute the values $u(ih, ih, t)$, the values $u(ih, ih, 0)$, $u(ih, ih, k)$, ... $u(ih, ih, t - k)$ are first computed and then used to determine the values of u at the t -level.

```
import math
from math import *
# NumPy, a numerical package for Python with linear algebra
# capabilities is imported
import numpy
from numpy import *

# The function Tridiagonal finds the solution vector Y to a
# tridiagonal matrix

def Tri(n,A,D,C,B,Y):

    # column vector X is initialized as empty

    X = empty([n,1])

    # the i-loop contains the procedure for solving the
    # tridiagonal matrix

    for i in range(2,n + 1,1):
        xmult = (float(A[i - 2:i - 1,0:1])
                 / float(D[i - 2:i - 1,0:1]))
        D[i - 1:i,0:1] = (float(D[i - 1:i,0:1])
                          - xmult * float(C[i - 2:i - 1,0:1]))
        B[i - 1:i,0:1] = (float(B[i - 1:i,0:1])
                          - xmult * float(B[i - 1:i,0:1]))

    X[n - 1:n,0:1] = (float(B[n - 1:n,0:1])
                      / float(D[n - 1:n,0:1]))

    for i in range(n - 1,0,-1):
        X[i - 1:i,0:1] = (float(B[i - 1:i,0:1])
                          - float(C[i - 1:i,0:1])
                          * float(X[i:i + 1,0:1]))
                          / float(D[i - 1:i,0:1]))

    return X

# f(x) returns the value u(x,y,0)
```



```

def f(x):
    y = 1
    return y

# The main function uses the Crank-Nicolson method to
# numerically solve the two-dimensional heat equation

def main(h,k,m,n,alpha):

    beta = k / (alpha * h ** 2)

    # column vectors D,C,U,V are initialized as empty

    D,C,U,V = empty([n,1]),empty([n,1]),empty([n,1]),
    empty([n,1])

    # column vectors D and C are assigned the values
    # along the diagonal bands of the coefficient matrix
    # the unknown column vector u is assigned the initial
    # values u(ih,ih,0)

    for i in range(1,n,1):
        D[i - 1:i,0:1] = 1
        C[i - 1:i,0:1] = beta
        U[i - 1:i,0:1] = f(i * h)

    for j in range(1,m + 1,1):
        for i in range(1,n,1):
            D[i - 1:i,0:1] = 1
            V[i - 1:i,0:1] = float(U[i - 1:i,0:1])

        Tri(n - 1,-C, D, C, V, V)

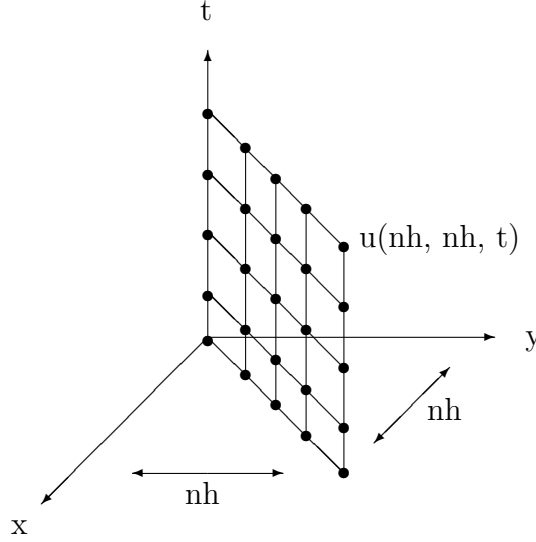
        for i in range(1,n,1):
            U[i - 1:i,0:1] = float(V[i - 1:i,0:1])

    return U

```

It should be emphasized that the `main` function only determines the values of u at the diagonal grid points (ih, ih, jk) , which corresponds to a planar lattice intersecting the xy -plane (Figure 2). The reason for this is because of the simplifying assumption made in equation (20), which eliminated the $u(x + h, y, t)$ and $u(x, y + h, t)$ terms.

Figure 2
Region of simulation



Assuming that the number of steps in the t variable is equivalent to the number of steps in the x and y variables, the run time for the `main` function is $O(n^2)$, since the `Tri` function involves $O(n)$ operations and is called by `main` $O(n)$ times. The run-time complexity of the Gaussian-elimination procedure in the `Tri` function is reduced greatly by virtue of the fact that the system being solved is sparse, i.e., mostly empty.

Theoretically, equation (19) can be used for numerically solving equation (7a) on the entire xyt lattice, as it contains the terms $u(x + h, y, t)$ and $u(x, y + h, t)$. On a given t level, these terms allow the x and y variables to be incremented by step-size h , so in principle u could be determined at each planar lattice parallel to the xy plane. The t variable could then be advanced with step-size k , covering the entire grid. This algorithm would require that the for-loop structure in `main` be nested in additional for-loops, and thus the number of operations for this algorithm would be at least $O(n^3)$. However, attempting to impliment this would greatly increase the complexity of the computer code, and hence it is not attempted.

Error Analysis and Conclusion

Since equation (21) was derived from intermediate expressions having errors $O(h)$, equation (21) must also have error $O(h)$, so we can expect the error to decrease linearly as the step size h is decreased. The Gaussian elimination process will not encounter zero divisors if the columns of the tridiagonal system (22) satisfy the diagonal dominance condition $|b_j| > |a_j| + |c_j|$ (Press, Teukolsky, et al.,). For equation (22), this condition becomes,

$$1 > 2\beta \implies k < \frac{\alpha h^2}{2}. \quad (23)$$

Hence for a modest value of h , we can expect some degree of numerical stability for a correspondingly much smaller range of k values. However, inequality (23) gives only a fairly weak result on the stability of the computer procedure used, and we can obtain stronger results by considering the condition number of the tridiagonal matrix. This is necessary to do if the tridiagonal system involves the empirically-determined constant α , and errors in α will effect the accuracy of the solution vector. Additionally, if the initial heat distribution is empirically determined, this will be a further source of error.

We can determine a lower bound for the condition number when $t = 0$. We assume throughout that the matrix norm is the l_2 matrix norm, given by

$$\|\mathbf{A}\|_2 = \sqrt{|\lambda_{max}|}, \quad (24)$$

where λ_{max} is the maximum eigenvalue of the matrix $\mathbf{A}^T \mathbf{A}$. The vector norm will be given by the standard Euclidean metric. To begin, we assume that the error vector $d\mathbf{b}$ is given by,

$$d\mathbf{b} = \begin{pmatrix} \sigma_0 \\ \sigma_0 \\ \vdots \\ \sigma_0 \end{pmatrix} \quad (25)$$

where σ_0 is the error in the measured initial heat distribution. We have that $\|d\mathbf{b}\| = \sqrt{n}\sigma_0$. It can be shown that

$$d\mathbf{a} = \mathbf{A}^{-1}d\mathbf{b}, \quad (26)$$

where $d\mathbf{a}$ is the error vector for $\mathbf{a}$. Using the properties of matrix norms, we have that,

$$\|d\mathbf{a}\| = \|\mathbf{A}^{-1}d\mathbf{b}\| \leq \|\mathbf{A}^{-1}\|_2 \|d\mathbf{b}\| = \sqrt{n}\sigma_0 \|\mathbf{A}^{-1}\|_2. \quad (27)$$

Therefore,

$$\frac{\|d\mathbf{a}\|}{\|d\mathbf{b}\|} \leq \|\mathbf{A}^{-1}\|_2 \implies \frac{\|d\mathbf{b}\|}{\|d\mathbf{a}\|} \geq \frac{1}{\|\mathbf{A}^{-1}\|_2}. \quad (28)$$

If the step-size k is sufficiently small, we can take the approximation $\|\mathbf{a}\| \approx \|\mathbf{b}\|$. If we use the well-known relationship,

$$\frac{\|d\mathbf{b}\|_2 \|\mathbf{a}\|}{\|d\mathbf{a}\|_2 \|\mathbf{b}\|} \leq \kappa(A), \quad (29)$$

we have that

$$\frac{1}{\|\mathbf{A}^{-1}\|_2} \leq \frac{\|d\mathbf{b}\|}{\|d\mathbf{a}\|} \leq \kappa(A). \quad (30)$$

We need a way of finding the value of $\|\mathbf{A}^{-1}\|_2$, i.e., the value $\sqrt{\mu_{max}}$ of the matrix $(\mathbf{A}^{-1})^T \mathbf{A}^{-1}$. By Gershgorin's theorem, the eigenvalues of $\mathbf{A}$ must lie inside a closed ball in $\mathbb{R}^1$, with radius 2β and center 1,

$$\lambda_i \in B(1; 2\beta). \quad (31)$$

Here, the closed ball identifies the interval $B(1; 2\beta) = \{x \mid d(x, 1) \leq 2\beta\}$, where $d(x, y)$ is the Euclidean metric. Hence we see that,

$$1 - 2\beta \leq \lambda_{min}, \lambda_{max} \leq 1 + 2\beta. \quad (32)$$

Using properties of eigenvalues, since λ is an eigenvalue of $\mathbf{A}$, λ^{-1} is an eigenvalue of $\mathbf{A}^{-1}$. Therefore,

$$\frac{1}{1 + 2\beta} \leq \lambda_{min}^{-1}, \lambda_{max}^{-1} \leq \frac{1}{1 - 2\beta}. \quad (33)$$

Since $\mathbf{A}^{-1}$ is a square matrix, it has the same eigenvalues as $(\mathbf{A}^{-1})^T$, and thus,

$$\frac{1}{1 + 2\beta} \leq (\lambda_{min}^{-1})^T, (\lambda_{max}^{-1})^T \leq \frac{1}{1 - 2\beta}. \quad (34)$$

As $\mathbf{A}^{-1}$ and $(\mathbf{A}^{-1})^T$ share the same set of eigenvalues and eigenvectors, the maximum eigenvalue of the matrix $(\mathbf{A}^{-1})^T \mathbf{A}^{-1}$, μ_{max} , must satisfy,

$$\mu_{max} = \lambda_{max}^{-1} (\lambda_{max}^{-1})^T \leq \frac{1}{(1 - 2\beta)^2}. \quad (35)$$

This implies that, because $\|\mathbf{A}^{-1}\|_2 = \sqrt{\mu_{max}}$,

$$1 - 2\beta \leq \frac{1}{\|\mathbf{A}^{-1}\|_2} \leq \kappa(A) \implies 1 - \frac{2k}{\alpha h^2} \leq \kappa(A). \quad (36)$$

Equation (36) suggests that the lower bound for the condition number is approximately 1 for $\alpha > 0$. Thus we can expect the tridiagonal system (22) to be well-conditioned when $t = 0$ and k is sufficiently small.

References

- [1] Cheney, Ward and Kincaid, David. *Numerical Mathematics and Computing*. Belmont: Thomson Brooks/Cole. Sixth Edition, 2004.
- [2] Kolman, Bernard and Hill, David R. *Elementary Linear Algebra with Applications* New Jersey: Pearson Education, Inc. Ninth edition, 2008.
- [3] Kasriel, Robert H. *Undergraduate Topology*. New York: Dover. First edition, 2009.
- [4] Press, W.H., Teukolsky, S.A., et al. *Numerical Recipes: The Art of Scientific Computing*. New York: Cambridge University Press. Third Edition, 2007.
- [5] Thoe, Dale W. and Zachmanoglou, E.C. *Introduction to Partial Differential Equations with Applications*. New York: Dover Publications, 1986.
- [6] Zelle, John. *Python Programming: An Introduction to Computer Science*. Franklin, Beedle and Associates Inc. Second Edition, 2010.